



Practical-4

Problem Definition: -

- Build a Node/Express backend for a Task Management system.
- Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet).
- Apply a request logging middleware that logs method, URL, and timestamp for every incoming request.
- Apply a global error handling middleware as the last middleware in the pipeline.
- Use correct HTTP status codes (200, 201, 404, 500) for each response.

Code:- Server.js:

```
const express = require('express');
const app = express();

let tasks = [
  { id: 1, title: 'Learn Node.js basics', completed: true },
  { id: 2, title: 'Build Express REST API', completed: false },
  { id: 3, title: 'Understand Express Middleware', completed: false }
];
let nextId = 4;

app.use(express.json());

// Request logger
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.url}`);
  next();
});

// Content-Type validator for POST/PUT
const requireJsonContent = (req, res, next) => {
  if (['POST', 'PUT'].includes(req.method)) {
    const contentType = req.headers['content-type'];
    if (!contentType || !contentType.includes('application/json')) {
      return res.status(400).json({
        error: 'Bad Request',
        message: 'Content-Type header must be application/json for POST and PUT requests'
      });
    }
  }
  next();
};
```

```

    });
  }
}
next();
});

app.use(requireJsonContent);
// ID validation middleware
const validateTaskId = (req, res, next) => {
  const idParam = req.params.id;
  const parsedId = parseInt(idParam, 10);

  if (isNaN(parsedId) || parsedId.toString() !== idParam || parsedId <= 0) {
    return res.status(400).json({
      error: 'Invalid Request',
      message: `Task ID '${idParam}' must be a positive integer`
    });
  }
  req.taskId = parsedId;
  next();
};

// CRUD Routes
app.get('/tasks', (req, res) => {
  res.status(200).json({ success: true, count: tasks.length, data: tasks });
});

app.get('/tasks/:id', validateTaskId, (req, res) => {
  const task = tasks.find(t => t.id === req.taskId);
  if (!task) {
    return res.status(404).json({ error: 'Not Found', message: `Task with ID ${req.taskId} not found` });
  }
  res.status(200).json({ success: true, data: task });
});

app.post('/tasks', (req, res) => {
  const { title, completed } = req.body;
  if (!title || typeof title !== 'string' || title.trim() === '') {
    return res.status(400).json({ error: 'Validation Error', message: 'Task title is required' });
  }
  const newTask = { id: nextId++, title: title.trim(), completed: !!completed };
  tasks.push(newTask);
  res.status(201).json({ success: true, message: 'Task created successfully', data: newTask });
});

app.put('/tasks/:id', validateTaskId, (req, res) => {
  const taskIndex = tasks.findIndex(t => t.id === req.taskId);
  if (taskIndex === -1) {
    return res.status(404).json({ error: 'Not Found', message: `Task with ID ${req.taskId} not found` });
  }
  const { title, completed } = req.body;
  if (title !== undefined && (typeof title !== 'string' || title.trim() === '')) {

```

```

    return res.status(400).json({ error: 'Validation Error', message: 'Task title must be non-
empty string' });
  }
  if (completed !== undefined && typeof completed !== 'boolean') {
    return res.status(400).json({ error: 'Validation Error', message: 'Completed must be a
boolean' });
  }

  tasks[taskIndex] = {
    ...tasks[taskIndex],
    ...(title !== undefined && { title: title.trim() }),
    ...(completed !== undefined && { completed })
  };
  res.status(200).json({ success: true, message: 'Task updated successfully', data:
tasks[taskIndex] });
});

app.delete('/tasks/:id', validateTaskId, (req, res) => {
  const taskIndex = tasks.findIndex(t => t.id === req.taskId);
  if (taskIndex === -1) {
    return res.status(404).json({ error: 'Not Found', message: `Task with ID ${req.taskId} not
found` });
  }
  const deletedTask = tasks.splice(taskIndex, 1)[0];
  res.status(200).json({ success: true, message: `Task ${req.taskId} deleted`, data:
deletedTask });
});

app.get('/cause-error', (req, res, next) => {
  next(new Error('Simulated internal server error'));
});

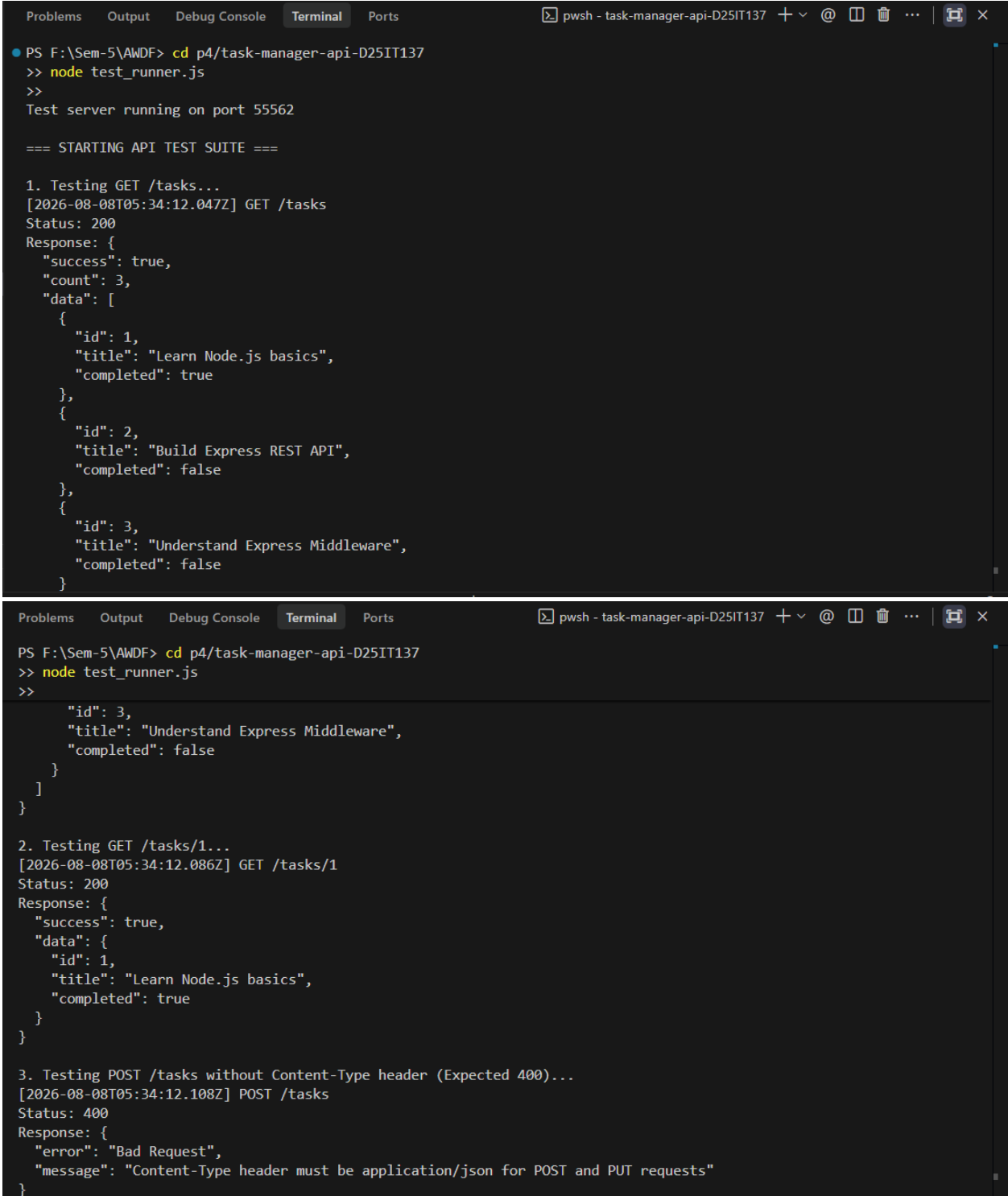
// 404 Handler
app.use((req, res) => {
  res.status(404).json({ error: 'Not Found', message: `Cannot ${req.method} ${req.url} -
Endpoint does not exist` });
});

// Global Error Handler
app.use((err, req, res, next) => {
  console.error(`[Error Logged]: ${err.stack}`);
  res.status(500).json({ error: 'Internal Server Error', message: 'Something went wrong on the
server' });
});

module.exports = app;
if (require.main === module) {
  const PORT = process.env.PORT || 5000;
  app.listen(PORT, () => {
    console.log(`Server listening on port ${PORT}`);
  });
}

```

Screenshots / Terminal Execution:-



The image displays two screenshots of a terminal window running a test suite. The terminal has tabs for Problems, Output, Debug Console, Terminal, and Ports. The title bar indicates the session is 'pwsh - task-manager-api-D25IT137'.

First Screenshot:

```
PS F:\Sem-5\AWDF> cd p4/task-manager-api-D25IT137
>> node test_runner.js
>>
Test server running on port 55562

=== STARTING API TEST SUITE ===

1. Testing GET /tasks...
[2026-08-08T05:34:12.047Z] GET /tasks
Status: 200
Response: {
  "success": true,
  "count": 3,
  "data": [
    {
      "id": 1,
      "title": "Learn Node.js basics",
      "completed": true
    },
    {
      "id": 2,
      "title": "Build Express REST API",
      "completed": false
    },
    {
      "id": 3,
      "title": "Understand Express Middleware",
      "completed": false
    }
  ]
}
```

Second Screenshot:

```
PS F:\Sem-5\AWDF> cd p4/task-manager-api-D25IT137
>> node test_runner.js
>>
    {
      "id": 3,
      "title": "Understand Express Middleware",
      "completed": false
    }
  ]
}

2. Testing GET /tasks/1...
[2026-08-08T05:34:12.086Z] GET /tasks/1
Status: 200
Response: {
  "success": true,
  "data": {
    "id": 1,
    "title": "Learn Node.js basics",
    "completed": true
  }
}

3. Testing POST /tasks without Content-Type header (Expected 400)...
[2026-08-08T05:34:12.108Z] POST /tasks
Status: 400
Response: {
  "error": "Bad Request",
  "message": "Content-Type header must be application/json for POST and PUT requests"
}
```

```
Problems Output Debug Console Terminal Ports pwsh - task-manager-api-D25IT137 + v @ [ ] [ ] ... [ ] [ ] x
PS F:\Sem-5\AWDF> cd p4/task-manager-api-D25IT137
>> node test_runner.js
>>
}
}

3. Testing POST /tasks without Content-Type header (Expected 400)...
[2026-08-08T05:34:12.108Z] POST /tasks
Status: 400
Response: {
  "error": "Bad Request",
  "message": "Content-Type header must be application/json for POST and PUT requests"
}

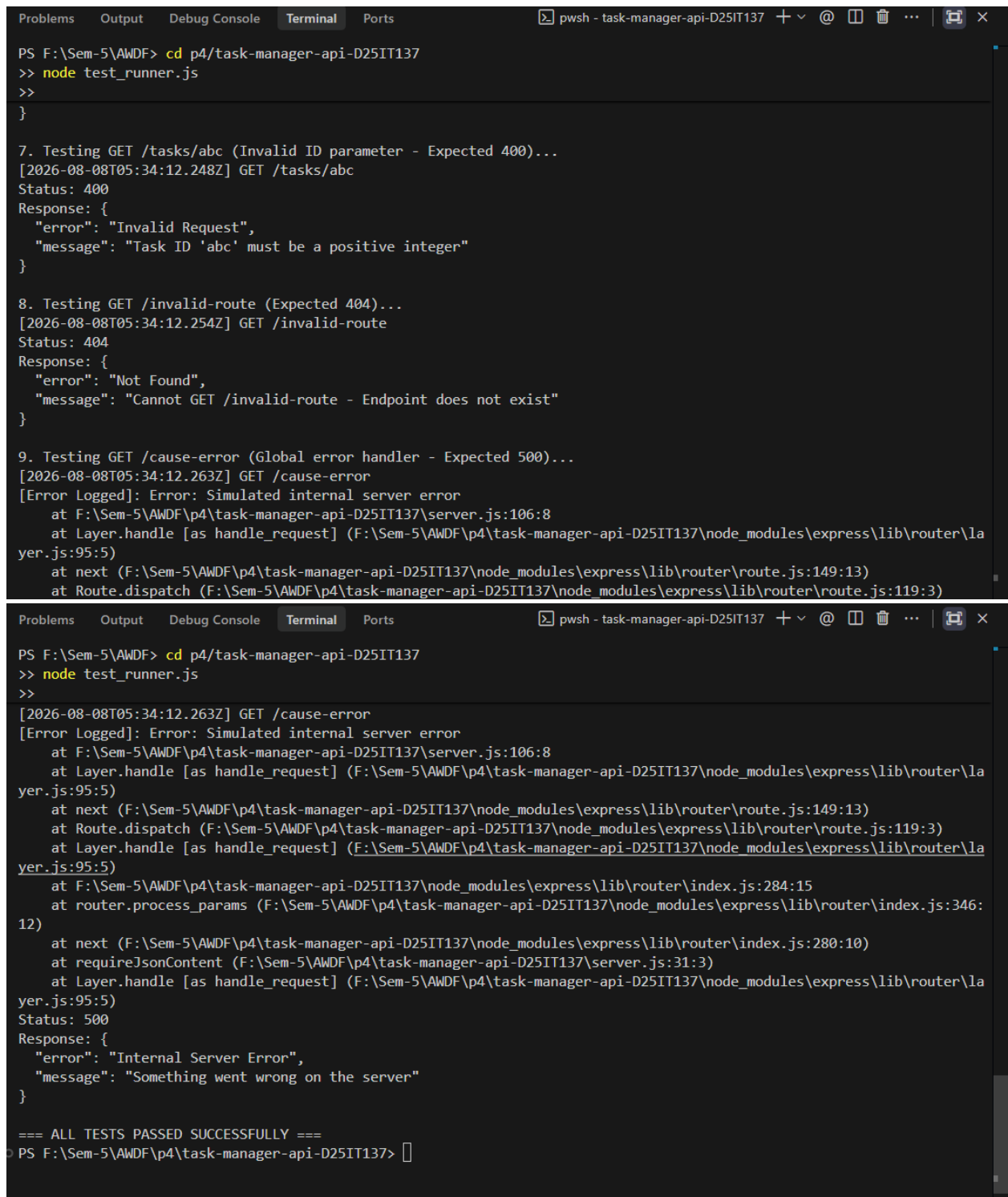
4. Testing POST /tasks (Create Task)...
[2026-08-08T05:34:12.210Z] POST /tasks
Status: 201
Response: {
  "success": true,
  "message": "Task created successfully",
  "data": {
    "id": 4,
    "title": "Complete Practical 4 Assignment",
    "completed": false
  }
}

5. Testing PUT /tasks/4 (Update Task)...
[2026-08-08T05:34:12.223Z] PUT /tasks/4
```

```
Problems Output Debug Console Terminal Ports pwsh - task-manager-api-D25IT137 + v @ [ ] [ ] ... [ ] [ ] x
PS F:\Sem-5\AWDF> cd p4/task-manager-api-D25IT137
>> node test_runner.js
>>
}

5. Testing PUT /tasks/4 (Update Task)...
[2026-08-08T05:34:12.223Z] PUT /tasks/4
Status: 200
Response: {
  "success": true,
  "message": "Task updated successfully",
  "data": {
    "id": 4,
    "title": "Complete Practical 4 Assignment",
    "completed": true
  }
}

6. Testing DELETE /tasks/2...
[2026-08-08T05:34:12.238Z] DELETE /tasks/2
Status: 200
Response: {
  "success": true,
  "message": "Task 2 deleted",
  "data": {
    "id": 2,
    "title": "Build Express REST API",
    "completed": false
  }
}
```



```

PS F:\Sem-5\AWDF> cd p4/task-manager-api-D25IT137
>> node test_runner.js
>>
}

7. Testing GET /tasks/abc (Invalid ID parameter - Expected 400)...
[2026-08-08T05:34:12.248Z] GET /tasks/abc
Status: 400
Response: {
  "error": "Invalid Request",
  "message": "Task ID 'abc' must be a positive integer"
}

8. Testing GET /invalid-route (Expected 404)...
[2026-08-08T05:34:12.254Z] GET /invalid-route
Status: 404
Response: {
  "error": "Not Found",
  "message": "Cannot GET /invalid-route - Endpoint does not exist"
}

9. Testing GET /cause-error (Global error handler - Expected 500)...
[2026-08-08T05:34:12.263Z] GET /cause-error
[Error Logged]: Error: Simulated internal server error
    at F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\server.js:106:8
    at Layer.handle [as handle_request] (F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\node_modules\express\lib\router\layer.js:95:5)
    at next (F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\node_modules\express\lib\router\route.js:149:13)
    at Route.dispatch (F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\node_modules\express\lib\router\route.js:119:3)

PS F:\Sem-5\AWDF> cd p4/task-manager-api-D25IT137
>> node test_runner.js
>>
[2026-08-08T05:34:12.263Z] GET /cause-error
[Error Logged]: Error: Simulated internal server error
    at F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\server.js:106:8
    at Layer.handle [as handle_request] (F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\node_modules\express\lib\router\layer.js:95:5)
    at next (F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\node_modules\express\lib\router\route.js:149:13)
    at Route.dispatch (F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\node_modules\express\lib\router\route.js:119:3)
    at Layer.handle [as handle_request] (F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\node_modules\express\lib\router\layer.js:95:5)
    at F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\node_modules\express\lib\router\index.js:284:15
    at router.process_params (F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\node_modules\express\lib\router\index.js:346:12)
    at next (F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\node_modules\express\lib\router\index.js:280:10)
    at requireJsonContent (F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\server.js:31:3)
    at Layer.handle [as handle_request] (F:\Sem-5\AWDF\p4\task-manager-api-D25IT137\node_modules\express\lib\router\layer.js:95:5)
Status: 500
Response: {
  "error": "Internal Server Error",
  "message": "Something went wrong on the server"
}

=== ALL TESTS PASSED SUCCESSFULLY ===
PS F:\Sem-5\AWDF\p4\task-manager-api-D25IT137>

```

Figure 1:- We can see that all Tests are passed successfully with all status codes.

Github Link:- [krunal9116/task-manager-api-D25IT137](https://github.com/krunal9116/task-manager-api-D25IT137)